

" به نام خدا "



دانشکده فنی مهندسی

رشته مهندسی کامپیوتر

گرایش هوش مصنوعی و نرم افزار

عنوان :

گزارش چهارم پروژه نهایی مباحث ویژه

دانشجویان :

محمد رضا باقرنژاد میری، یاسمن دادفر، کیارش رحمانی، معصومه مختاری

استاد :

دکتر مهدیه قاسمی

فهرست

۱.مقدمه	۳
۲. روش انجام کار	۵
۱-۲. زیرساخت CI/CD و کانتینرسازی	۶
۲-۲. سیستم تشخیص تغییر توزیع داده	۷
۳-۲. سیستم جمع‌آوری بازخورد و پایش آنلاین عملکرد مدل	۸
۴-۲. سیستم هشداردهی و پایش رویدادها	۹
۵-۲. سیستم بازآموزی خودکار مدل	۱۰
۶-۲. سیستم گزارش‌گیری نهایی و یکپارچه‌سازی پایپ‌لاین	۱۲
۷-۲. نتایج اجرای سیستم در محیط Docker و پایش Drift	۱۳
۳. نتیجه‌گیری	۱۶

١. مقدمه

در این فاز از پروژه، تمرکز اصلی تیم بر طراحی و پیاده‌سازی یک سیستم جامع پایش و نگهداشت مدل در چارچوب MLOps قرار گرفت؛ سیستمی که بتواند پس از استقرار مدل در محیط عملیاتی، به صورت مداوم رفتار آن را تحت نظر قرار داده و در برابر تغییرات داده و کاهش کیفیت عملکرد، واکنش مناسب نشان دهد. هدف از این مرحله، حرکت از یک مدل ایستا به سمت یک سیستم هوشمند و خودپایش بود که بتواند شرایط واقعی دنیای تولید را شبیه‌سازی کند؛ جایی که توزیع داده‌ها ثابت نیست و تغییرات تدریجی یا ناگهانی می‌توانند عملکرد مدل را به‌طور جدی تحت تأثیر قرار دهند.

در این راستا، یک زیرساخت یکپارچه برای پایش انواع مختلف تغییر در داده طراحی و پیاده‌سازی شد که شامل بررسی تغییر در توزیع ویژگی‌های ورودی، تغییر در توزیع برجسب‌ها، تغییر در رابطه میان ورودی و خروجی مدل و همچنین تغییرات ترکیبی در دامنه داده می‌شود. این زیرساخت به گونه‌ای توسعه داده شد که بتواند به صورت مستمر داده‌های جدید را با داده‌های مرجع مقایسه کرده و در صورت مشاهده انحراف معنادار، آن را به عنوان نشانه‌ای از افت احتمالی عملکرد مدل شناسایی کند.

همچنین در این مرحله، مکانیزم‌هایی برای پایش آنلاین عملکرد مدل و تحلیل رفتار آن در شرایط واقعی طراحی شد تا بتوان کیفیت پیش‌بینی‌ها را در قالب داده‌های بازخوردی کاربران ارزیابی کرد. در کنار این موضوع، یک چرخه خودکار برای تصمیم‌گیری در خصوص نیاز به بازآموزی مدل نیز پیاده‌سازی شد که بر اساس میزان تغییرات داده و افت عملکرد، فرآیند retraining را به صورت خودکار فعال می‌کند.

در نهایت، این بخش از پروژه به عنوان لایه نظارتی اصلی سیستم عمل کرده و زیرساختی فراهم کرده است که امکان حرکت به سمت یک سیستم یادگیری ماشین پایدار، قابل اعتماد و سازگار با تغییرات محیطی را فراهم می‌سازد. این فاز همچنین نقش کلیدی در تکمیل چرخه کامل MLOps شامل پایش، ارزیابی، تصمیم‌گیری و به‌روزرسانی مدل ایفا می‌کند.

۲. روش انجام کار

۱-۲. زیرساخت CI/CD و کانتینرسازی

در این بخش از پروژه، تیم اقدام به طراحی و پیاده‌سازی یک زیرساخت یکپارچه برای یکپارچه‌سازی فرآیند توسعه، تست و استقرار مدل‌های یادگیری ماشین در قالب یک سیستم CI/CD مبتنی بر GitHub Actions نمود. هدف اصلی از این مرحله، ایجاد یک لایه کنترل کیفیت خودکار و قابل اعتماد بود که بتواند در هر تغییر کد، وضعیت سلامت سیستم را بررسی کرده و از ورود کدهای ناسازگار به محیط اصلی جلوگیری کند.

در این راستا، تمامی اجزای سیستم در قالب یک محیط کانتینری شده مبتنی بر Docker اجرا شدند تا یک محیط ایزوله و بازتولیدپذیر فراهم شود. این تصمیم باعث شد وابستگی‌های نرم‌افزاری، نسخه کتابخانه‌ها و تنظیمات محیطی به صورت کامل کنترل شده و اختلاف میان محیط توسعه و محیط عملیاتی به حداقل برسد. به این ترتیب، رفتار مدل در تمامی مراحل توسعه و تست در شرایطی نزدیک به محیط production شبیه‌سازی شد. در لایه CI/CD، فرآیندهای خودکارسازی شده با استفاده از GitHub Actions پیاده‌سازی گردیدند که در فایل تنظیماتی CI، به عنوان نگهبان کیفیت سیستم عمل می‌کنند. این pipeline به گونه‌ای طراحی شد که در هر Push یا Pull Request، ابتدا استانداردهای کدنویسی و کیفیت ساختار با استفاده از ابزارهای linting بررسی شده و سپس تست‌های واحد و تست‌های یکپارچه‌سازی به صورت خودکار اجرا شوند. این موضوع نقش مهمی در جلوگیری از ورود خطاهای ساختاری و منطقی به کد اصلی داشته است.

در ادامه، مجموعه‌ای از تست‌های سلامت سیستم^۱ نیز به این فرآیند اضافه شد تا اطمینان حاصل شود که مازول‌های اصلی پروژه بدون خطا بارگذاری شده و ساختار داده‌ها با انتظار سیستم همخوانی دارد. نتایج اجرای تمامی این مراحل به صورت گزارش‌های قابل دسترس ذخیره شده و در اختیار تیم توسعه قرار می‌گیرد تا شفافیت کامل در فرآیند توسعه حفظ شود.

^۱ Smoke Tests

در مجموع، این زیرساخت CI/CD و کانتینرسازی به عنوان اولین لایه از معماری MLOps پروژه عمل کرده و نقش اساسی در تضمین پایداری، تکرارپذیری و کیفیت سیستم در چرخه توسعه مدل‌های یادگیری ماشین ایفا می‌کند.

۲-۲. سیستم تشخیص تغییر توزیع داده^۲

در این مرحله از پروژه، یک سیستم جامع برای پایش و تشخیص تغییرات در توزیع داده طراحی و پیاده‌سازی شد که هدف اصلی آن، شناسایی کاهش تدریجی یا ناگهانی کیفیت داده‌های ورودی و تأثیر آن بر عملکرد مدل در محیط عملیاتی است. این سیستم به عنوان یکی از هسته‌های اصلی MLOps Monitoring Layer عمل می‌کند و نقش مهمی در حفظ پایداری مدل در طول زمان دارد.

در طراحی این بخش، چهار نوع اصلی از تغییرات داده مورد بررسی قرار گرفت که شامل تغییر در ویژگی‌های ورودی^۳، تغییر در توزیع برچسب‌ها^۴، تغییر در رابطه میان ورودی و خروجی مدل^۵ و در نهایت تغییرات ترکیبی در دامنه داده^۶ می‌باشد. هر یک از این موارد به صورت مستقل مدل‌سازی و سپس در قالب یک pipeline یکپارچه ترکیب شدند تا امکان تحلیل جامع وضعیت داده فراهم شود.

برای تشخیص تغییر در ویژگی‌های ورودی، از آزمون‌های آماری مبتنی بر توزیع مانند KS-test بر روی طول متن‌ها استفاده شد تا اختلاف معنادار میان داده‌های آموزش و داده‌های جدید شناسایی شود. در بخش مربوط به

^۲ Drift Detection System

^۳ Covariate Shift

^۴ Label Shift

^۵ Concept Drift

^۶ Domain Shift

تغییر توزیع برچسب‌ها نیز اختلاف بین توزیع کلاس‌ها در داده‌های مرجع و داده‌های جدید محاسبه شده و به عنوان شاخصی از جابه‌جایی داده‌ها مورد استفاده قرار گرفت.

در بخش تشخیص تغییر مفهومی، رفتار مدل در مواجهه با داده‌های جدید مورد بررسی قرار گرفت و اختلاف میان میزان اطمینان مدل در پیش‌بینی‌های صحیح و نادرست به عنوان شاخصی از افت کیفیت رابطه یادگیری تحلیل شد. این رویکرد امکان شناسایی تغییراتی را فراهم می‌کند که صرفاً در سطح داده قابل مشاهده نیستند، بلکه در سطح رفتار مدل رخ می‌دهند.

در نهایت، تمامی این مؤلفه‌ها در قالب یک سیستم یکپارچه جمع شدند تا امکان تشخیص همزمان انواع مختلف drift فراهم شود و در صورت مشاهده تغییرات معنادار، سیگنال لازم برای سایر بخش‌های سیستم مانند alerting و retraining تولید گردد. این زیرساخت نقش کلیدی در تبدیل سیستم از یک مدل ایستا به یک سیستم هوشمند و سازگار با تغییرات محیطی ایفا می‌کند.

۲-۳. سیستم جمع‌آوری بازخورد و پایش آنلاین عملکرد مدل^۷

در این مرحله از پروژه، یک زیرساخت برای جمع‌آوری بازخورد از پیش‌بینی‌های مدل و تحلیل عملکرد آن در شرایط واقعی طراحی و پیاده‌سازی شد. هدف اصلی این بخش، ایجاد یک حلقه بازخوردی^۸ بود تا مدل بتواند نه تنها بر اساس داده‌های ایستا، بلکه بر اساس داده‌های تولیدشده در محیط عملیاتی نیز مورد ارزیابی قرار گیرد. در این راستا، مکانیزمی برای ثبت تمامی پیش‌بینی‌های مدل به همراه اطلاعات تکمیلی شامل متن ورودی، برچسب واقعی، برچسب پیش‌بینی‌شده و میزان اطمینان مدل طراحی شد. این داده‌ها به صورت ساختارمند در قالب فایل‌های JSONL ذخیره شده و به عنوان منبع اصلی برای تحلیل‌های آنلاین مورد استفاده قرار گرفتند.

^۷ Feedback & Online Metrics System

^۸ Feedback Loop

در ادامه، یک سیستم پایش آنلاین عملکرد مدل بر اساس مفهوم sliding window پیاده‌سازی شد تا بتوان دقت مدل را در بازه‌های زمانی اخیر مورد بررسی قرار داد. این رویکرد امکان تشخیص سریع افت عملکرد مدل در شرایط واقعی را فراهم می‌سازد، به گونه‌ای که مدل تنها بر اساس عملکرد تاریخی کلی ارزیابی نمی‌شود، بلکه رفتار آن در داده‌های اخیر نیز به طور مستقل تحلیل می‌گردد.

همچنین در این بخش، داده‌های بازخوردی کاربران به عنوان ورودی سیستم پایش استفاده شدند تا امکان محاسبه معیارهایی مانند دقت آنلاین فراهم شود. این موضوع باعث شد سیستم از حالت صرفاً آفلاین خارج شده و به یک سیستم پایش پویا و نزدیک به شرایط واقعی تولید تبدیل شود.

در نهایت، این زیرساخت به عنوان یکی از اجزای کلیدی حلقه یادگیری مداوم در سیستم MLOps عمل می‌کند و نقش مهمی در اتصال رفتار مدل در محیط واقعی به فرآیندهای ارزیابی، تشخیص تغییر و تصمیم‌گیری برای به‌روزرسانی مدل ایفا می‌نماید.

۲-۴. سیستم هشداردهی و پایش رویدادها^۹

در این مرحله از پروژه، یک سیستم هشداردهی مبتنی بر رویداد طراحی و پیاده‌سازی شد که هدف آن ثبت، تحلیل و اطلاع‌رسانی تغییرات مهم در رفتار مدل و داده‌ها در محیط عملیاتی است. این سیستم به عنوان لایه مشاهده‌پذیری^{۱۰} در معماری MLOps عمل کرده و نقش کلیدی در شفاف‌سازی وضعیت سیستم در زمان اجرا دارد.

در این زیرساخت، رویدادهای مهمی که در طول اجرای سیستم رخ می‌دهند به صورت ساختارمند ثبت و طبقه‌بندی می‌شوند. این رویدادها شامل تشخیص تغییر در توزیع داده، افت عملکرد مدل، و همچنین وقوع فرآیند

^۹ Alerting System & Monitoring Events

^{۱۰} Observability Layer

بازآموزی مدل هستند. هر رویداد همراه با اطلاعاتی مانند نوع هشدار، سطح اهمیت^{۱۱}، زمان وقوع و جزئیات مرتبط ذخیره می‌شود تا امکان تحلیل دقیق رفتار سیستم در طول زمان فراهم گردد.

در طراحی این بخش، تلاش شده است تا سیستم هشداردهی به صورت همزمان دو وظیفه اصلی را انجام دهد. از یک طرف، هشدارها به صورت لحظه‌ای در کنسول نمایش داده می‌شوند تا توسعه‌دهنده بتواند وضعیت سیستم را در زمان اجرا مشاهده کند، و از طرف دیگر، تمامی هشدارها در قالب داده‌های ساختاریافته ذخیره می‌شوند تا در مراحل بعدی بتوان از آنها برای تحلیل‌های آماری و بررسی روند عملکرد سیستم استفاده کرد.

همچنین این سیستم به گونه‌ای طراحی شده است که بتواند با سایر اجزای پروژه مانند سیستم تشخیص تغییر داده، متریک‌های آنلاین و فرآیند بازآموزی مدل یکپارچه شود. به این ترتیب، هرگونه تغییر معنادار در رفتار مدل یا داده‌ها به صورت خودکار به یک رویداد قابل ردیابی تبدیل شده و در چرخه پایش سیستم ثبت می‌شود.

در نهایت، این لایه هشداردهی نقش مهمی در تبدیل سیستم از یک مدل ساده یادگیری ماشین به یک سیستم قابل مشاهده، قابل پایش و قابل مدیریت در محیط عملیاتی ایفا می‌کند و پایه‌ای برای پیاده‌سازی معماری‌های پیشرفته‌تر مانند مانیتورینگ بلادرنگ و داشبوردهای تحلیلی فراهم می‌سازد.

۲-۵. سیستم بازآموزی خودکار مدل^{۱۲}

در این مرحله از پروژه، مکانیزمی برای بازآموزی خودکار مدل یادگیری ماشین طراحی و پیاده‌سازی شد که هدف آن، حفظ عملکرد مدل در شرایطی است که داده‌های ورودی دچار تغییرات معنادار می‌شوند. این سیستم به عنوان

^{۱۱} Severity

^{۱۲} Auto Retraining System

بخش تصمیم‌گیرنده در چرخه MLOps عمل کرده و مسئول فعال‌سازی فرآیند به‌روزرسانی مدل در صورت نیاز می‌باشد.

در طراحی این بخش، شرط اصلی فعال‌سازی فرآیند بازآموزی بر اساس خروجی سیستم تشخیص تغییر داده تعریف شد. به این صورت که در صورت مشاهده تغییر در توزیع داده‌های ورودی یا تغییر در توزیع برچسب‌ها، سیستم به صورت خودکار تصمیم به آغاز فرآیند retraining می‌گیرد. این تصمیم‌گیری به گونه‌ای انجام شده است که تنها تغییرات معنادار و قابل توجه منجر به اجرای مجدد آموزش مدل شوند تا از بازآموزی‌های غیرضروری جلوگیری گردد.

پس از فعال شدن این فرآیند، مدل بر اساس داده‌های جدید مجدداً آموزش داده شده و نسخه به‌روزرسانی شده آن جایگزین مدل قبلی می‌شود. این فرآیند به صورت کاملاً خودکار در داخل pipeline سیستم اجرا شده و نیازی به مداخله دستی ندارد، که این موضوع یکی از اهداف اصلی طراحی سیستم‌های MLOps در مقیاس عملیاتی محسوب می‌شود.

علاوه بر این، وضعیت اجرای فرآیند بازآموزی نیز به عنوان یک رویداد مستقل در سیستم ثبت شده و در بخش گزارش‌گیری نهایی مورد استفاده قرار می‌گیرد. این موضوع امکان ردیابی تغییرات مدل در طول زمان و تحلیل اثر بازآموزی بر عملکرد سیستم را فراهم می‌سازد.

در نهایت، این زیرسیستم نقش حیاتی در تبدیل مدل از یک سیستم ایستا به یک سیستم پویا و خودسازگار ایفا می‌کند و امکان تطبیق مداوم مدل با تغییرات محیطی را در چارچوب یک معماری MLOps واقعی فراهم می‌سازد.

۲-۶. سیستم گزارش‌گیری نهایی و یکپارچه‌سازی پایپ‌لاین

در این مرحله از پروژه، یک سیستم گزارش‌گیری نهایی طراحی و پیاده‌سازی شد که هدف آن تجمیع تمامی خروجی‌های بخش‌های مختلف سیستم پایش مدل در قالب یک ساختار یکپارچه و قابل تحلیل است. این سیستم به عنوان نقطه نهایی چرخه MLOps Monitoring عمل کرده و نقش جمع‌بندی‌کننده وضعیت کلی مدل در محیط عملیاتی را بر عهده دارد.

در این زیرسیستم، نتایج حاصل از بخش‌های مختلف شامل تشخیص تغییر در داده، متریک‌های عملکردی، ارزیابی آفلاین و آنلاین، وضعیت بازخوردهای جمع‌آوری‌شده و همچنین اطلاعات مربوط به فرآیند بازآموزی مدل در یک ساختار واحد ترکیب می‌شوند. این تجمیع داده‌ها امکان تحلیل جامع رفتار سیستم را در یک بازه زمانی مشخص فراهم می‌کند.

علاوه بر این، یک شاخص سلامت کلی برای سیستم تعریف شده است که بر اساس میزان تغییرات داده و افت عملکرد مدل محاسبه می‌شود. این شاخص به عنوان معیار اصلی برای ارزیابی وضعیت فعلی مدل مورد استفاده قرار می‌گیرد و سیستم را در یکی از وضعیت‌های پایدار، هشدار یا بحرانی طبقه‌بندی می‌کند. این طبقه‌بندی نقش مهمی در تصمیم‌گیری‌های بعدی سیستم از جمله نیاز به بازآموزی یا بررسی بیشتر ایفا می‌کند.

در ادامه، کل فرآیند پایش به صورت یک pipeline یکپارچه سازمان‌دهی شده است که در آن مراحل تشخیص تغییر، ثبت هشدارها، اجرای بازآموزی، محاسبه متریک‌ها و تولید گزارش نهایی به صورت پشت سر هم و خودکار اجرا می‌شوند. این یکپارچگی باعث شده است سیستم از حالت ماژول‌های مستقل خارج شده و به یک چرخه کامل و پیوسته MLOps تبدیل شود.

در نهایت، خروجی این بخش به صورت یک فایل گزارش ساختارمند ذخیره می‌شود که شامل تمامی اطلاعات کلیدی مربوط به وضعیت مدل و عملکرد سیستم است و می‌تواند به عنوان مبنای تحلیل، مانیتورینگ و تصمیم‌گیری در مراحل بعدی توسعه مورد استفاده قرار گیرد.

۷-۲. نتایج اجرای سیستم در محیط Docker و پایش Drift

در این بخش، عملکرد سیستم پیاده‌سازی شده در محیط Docker مورد ارزیابی قرار گرفت تا رفتار مدل و زیرساخت پایش در شرایط شبه‌تولید^{۱۳} بررسی شود. این مرحله با هدف اعتبارسنجی عملی سیستم تشخیص تغییر داده، متریک‌های عملکردی و API سرویس پیش‌بینی انجام شد.

در اولین آزمایش، سرویس پیش‌بینی مدل از طریق API داخلی در محیط Docker اجرا شد و مجموعه‌ای از نمونه‌های متنی به سیستم ارسال گردید. نتایج نشان داد که مدل قادر به پاسخ‌دهی پایدار در قالب پیش‌بینی احساسات^{۱۴} بوده و مقادیر احتمال برای کلاس‌های مثبت و منفی به صورت معتبر تولید شده‌اند. همچنین میانگین تأخیر پاسخ‌دهی^{۱۵} در حدود ۱.۵ میلی‌ثانیه ثبت شد که نشان‌دهنده عملکرد مناسب سرویس در شرایط بار سبک می‌باشد.

در بخش پایش سیستم، endpoint مربوط به metrics وضعیت کلی سیستم را گزارش کرد. بر اساس خروجی ثبت‌شده، تعداد درخواست‌ها و پیش‌بینی‌ها برابر بوده و سیستم دارای ۱۱ درخواست موفق بوده است. همچنین هیچ داده feedback ثبت نشده و میانگین latency برابر ۱.۵۵ میلی‌ثانیه گزارش شده است. این نتایج نشان‌دهنده پایداری سرویس در سطح اولیه استقرار می‌باشد.

^{۱۳} production-like environment

^{۱۴} sentiment classification

^{۱۵} latency

در بخش تشخیص تغییر داده^{۱۶}، سیستم با استفاده از معیار PSI^{۱۷} تغییرات ویژگی‌های کلیدی داده را بررسی کرد. نتایج نشان داد که در تمامی ویژگی‌های مهم شامل طول کاراکتر^{۱۸}، تعداد کلمات^{۱۹} و میانگین طول کلمات^{۲۰}، مقدار PSI به صورت قابل توجهی بالا بوده و وضعیت همه ویژگی‌ها در سطح “Significant Shift” طبقه‌بندی شده‌اند. به عنوان مثال، مقدار PSI برای ویژگی word_count برابر ۱۲.۴۵۷ و برای char_length برابر ۱۲.۴۳۳ ثبت شد که نشان‌دهنده تغییر شدید توزیع داده‌های ورودی در محیط تولید نسبت به داده مرجع است. همچنین endpoint مربوط به drift-check نیز این نتایج را تأیید کرده و وضعیت کلی سیستم را در حالت “Significant Shift” گزارش کرده است. در این حالت، تعداد داده‌های مرجع برابر ۴۰۰۰ و داده‌های تولیدی برابر ۱۲ بوده است که اختلاف شدید بین این دو مجموعه داده را نشان می‌دهد. این موضوع تأیید می‌کند که سیستم تشخیص drift به درستی فعال شده و قادر به شناسایی تغییرات توزیع داده در محیط عملیاتی می‌باشد. در مجموع، نتایج اجرای سیستم در محیط Docker نشان داد که زیرساخت طراحی شده نه تنها قادر به ارائه پیش‌بینی‌های پایدار است، بلکه به صورت همزمان توانایی پایش رفتار داده، تشخیص تغییر توزیع و ارائه گزارش‌های تحلیلی را نیز دارد. این موضوع اعتبار طراحی MLOps pipeline را در شرایط نزدیک به محیط واقعی تولید تأیید می‌کند.

^{۱۶} Drift Detection

^{۱۷} Population Stability Index

^{۱۸} char_length

^{۱۹} word_count

^{۲۰} avg_word_length

```

PS C:\Users\Kiarash Rahmani\Desktop\Codes\TMU\Sem2\LLM\Project\sentiment-analysis-mlops> docker compose run --rm api python drift_detector.py
[+] Creating 1/1
Container sentiment-mflow Running 0.0s
{
  "status": "Significant shift",
  "reference_rows": 4000,
  "production_rows": 12,
  "thresholds": {
    "no_change": "PSI < 0.1",
    "moderate_shift": "0.1 <= PSI <= 0.25",
    "significant_shift": "PSI > 0.25"
  },
  "features": [
    {
      "feature": "char_length",
      "psi": 12.43386,
      "status": "Significant shift"
    },
    {
      "feature": "word_count",
      "psi": 12.457941,
      "status": "Significant shift"
    },
    {
      "feature": "avg_word_length",
      "psi": 9.082867,
      "status": "Significant shift"
    }
  ]
}
PS C:\Users\Kiarash Rahmani\Desktop\Codes\TMU\Sem2\LLM\Project\sentiment-analysis-mlops>

```

شکل ۱-۲ خروجی اجرای Drift Detection در محیط Docker

```

>> "i loved the acting and the story",
>> "terrible movie and boring plot",
>> "waste of time i hated it",
>> "the film was emotional and beautifully made",
>> "bad acting and weak dialogue",
>> "fantastic soundtrack and wonderful cast",
>> "awful script with no real direction"
>> }
PS C:\Users\Kiarash Rahmani\Desktop\Codes\TMU\Sem2\LLM\Project\sentiment-analysis-mlops>
PS C:\Users\Kiarash Rahmani\Desktop\Codes\TMU\Sem2\LLM\Project\sentiment-analysis-mlops> foreach ($text in $texts) {
>>   Invoke-RestMethod `
>>     -Uri "http://localhost:8000/predict" `
>>     -Method POST `
>>     -ContentType "application/json" `
>>     -Body (@{ text = $text } | ConvertTo-Json)
>> }

sentiment probabilities                                     latency_ms
-----
positive @{negative=0.12310673361306756; positive=0.8768932663869324} 1.776
positive @{negative=0.129067318459668; positive=0.870932681540332} 1.275
negative @{negative=0.995484808028621; positive=0.004515191971378904} 1.163
negative @{negative=0.9957015086525481; positive=0.004298491347451884} 1.638
positive @{negative=0.055088621195434895; positive=0.9449113788045651} 1.211
negative @{negative=0.9939809742851835; positive=0.006019025714816516} 1.228
positive @{negative=0.0031099962258688274; positive=0.9968900037741312} 1.178
negative @{negative=0.9955000551731873; positive=0.004499944826812694} 1.24

PS C:\Users\Kiarash Rahmani\Desktop\Codes\TMU\Sem2\LLM\Project\sentiment-analysis-mlops>

```

شکل ۲-۲ خروجی مدل sentiment analysis شامل پیش‌بینی احساسات، احتمال کلاس‌ها و latency در محیط اجرا

۳. نتیجه‌گیری

در این فاز از پروژه، یک سیستم کامل و یکپارچه MLOps برای پایش، ارزیابی و نگهداشت مدل یادگیری ماشین طراحی و پیاده‌سازی شد که توانایی مدیریت چرخه کامل عملکرد مدل در محیط عملیاتی را فراهم می‌کند. این سیستم با ترکیب اجزایی مانند تشخیص تغییر توزیع داده، پایش آنلاین عملکرد، جمع‌آوری بازخورد، سیستم هشداردهی و بازآموزی خودکار، یک معماری پویا و نزدیک به شرایط واقعی تولید را شبیه‌سازی کرده است.

نتایج به‌دست‌آمده از اجرای سیستم در محیط Docker نشان داد که مدل قادر است در شرایط سرویس‌دهی واقعی، پیش‌بینی‌هایی با latency پایین و عملکرد پایدار ارائه دهد. در کنار آن، سیستم تشخیص تغییر داده توانست به صورت دقیق تغییرات شدید در توزیع ویژگی‌های ورودی را شناسایی کرده و وضعیت سیستم را در سطح “Significant Shift” گزارش دهد. این موضوع نشان‌دهنده حساسیت و صحت عملکرد لایه پایش داده در مواجهه با تغییرات محیطی است.

همچنین مکانیزم بازآموزی خودکار بر اساس خروجی سیستم تشخیص تغییر فعال شده و توانست بدون نیاز به مداخله دستی، مدل را در شرایط تغییر داده به‌روزرسانی کند. این ویژگی نقش مهمی در افزایش پایداری و تطبیق‌پذیری سیستم در طول زمان ایفا می‌کند. علاوه بر این، سیستم جمع‌آوری بازخورد و متریک‌های آنلاین امکان ارزیابی مستمر عملکرد مدل را در شرایط واقعی فراهم کرده و دید دقیقی از کیفیت پیش‌بینی‌ها در طول زمان ارائه می‌دهد.

در مجموع، این فاز از پروژه نشان داد که می‌توان با ترکیب اجزای مختلف MLOps، یک چرخه کامل شامل توسعه، استقرار، پایش، تشخیص تغییر و بازآموزی خودکار ایجاد کرد که قادر است در برابر تغییرات داده مقاوم بوده و عملکرد پایدار مدل را در محیط عملیاتی تضمین کند.